# EEEN30160: Lab Assignment 4

## Filter Design, Evaluation and application

---

MATLAB has a variety of functions in its signal processing toolbox which support the design and analysis of analog and digital filters. We shall begin with some prototype analog filter designs and take a look at the most commonly used digital IIR approximations. Specifically we will examine:

- Butterworth filters
- Chebyshev filters
- Elliptic filters

These filters are typically specified by their cutoff frequencies and ripples on both the stopband and passband. The first step is to determine the minimum order and cutoff frequency of the filter required to meet the desired specifications. The second step is to pass this order and cutoff frequency to the design functions butter, cheby1, cheby2, or ellip. The following commands give estimates of the filter's order and cutoff frequency:

```
[N,Wc]=buttord(Wp,Ws, Rp,Rs);   %Butterworth filter
[N,Wc]=cheb1ord(Wp,Ws,Rp,Rs);   %Chebyshev Type-I
[N,Wc]=cheb2ord(Wp,Ws,Rp,Rs);   %Chebyshev Type-II
[N,Wc]=ellipord(Wp,Ws,Rp,Rs);   %Elliptic
```

N = minimum order of the digital IIR filter to meet specifications

Wc = cutoff frequency

Rp = maximum passband ripple in dB

Rs = minimum stopband attenuation in dB – given as positive number even though attenuation corresponds to negative dB values.

Wp,Ws = passband and stopband edges frequencies, normalized relative to half the sample rate (i.e. Nyquist frequency), so 0<Wp,Ws<1 where 1 corresponds to $f_s/2$.

**Example 1:**

Find the order and cutoff frequency of a Butterworth digital lowpass filter with no more than 1 dB attenuation below 3400 Hz, and at least 40 dB attenuation at and beyond 3600 Hz. The sampling frequency is 8000 Hz.

```
 Fs=8000; %specify sampling rate
Fn=Fs/2; %specify Nyquist frequency
Wp=3400/Fn; %normalized passband frequency
Ws=3600/Fn; %normalized stopband frequency
Rp=1; Rs=40; %passband and stopband attenuation
[N,Wc]=buttord(Wp,Ws,Rp,Rs)
```

We shall now consider two techniques for completing the design of IIR filters – i.e., determining the numerator and denominator coefficients of the Z-domain transfer function, b and a respectively. The signal processing toolbox includes butter, cheby1, cheby2, and ellip functions that automate the design of IIR filters via the bilinear transformation method. Here are a list of examples with explanations in the comments:

```
[b,a]=butter(n,wn); %LPF: 0<wn<1
[b,a]=butter(n,wn,'high'); %HPF: 0<wn<1
[b,a]=butter(n,wn); %BPF: wn=[w1,w2], 0<w1,w2<1
[b,a]=butter(n,wn,'stop'); %BSF: wn=[w1,w2],0<w1,w2<1
[b,a]=cheby1(n,Rp,wn); %LPF: 0<wn<1
[b,a]=cheby2(n,Rs,wn); %LPF: 0<wn<1
[b,a]=cheby1(n,Rp,wn,'high'); %HPF: 0<wn<1
[b,a]=cheby2(n,Rs,wn,'high'); %HPF: 0<wn<1
[b,a]=cheby1(n,Rp,wn,'stop'); %BSF: wn=[w1,w2], 0<w1,w2<1
[b,a]=cheby2(n,Rs,wn,'stop'); %BSF: wn=[w1,w2], 0<w1,w2<1
[b,a]=cheby1(n,Rp,wn); %BPF: wn=[w1,w2], 0<w1,w2<1
[b,a]=cheby2(n,Rs,wn); %BPF: wn=[w1,w2], 0<w1,w2<1
[b,a]=ellip(n,Rp,Rs,wn); %LPF: 0<wn<1
[b,a]=ellip(n,Rp,Rs,wn,'high'); %HPF: 0<wn<1
[b,a]=ellip(n,Rp,Rs,wn,'stop'); %BSF: wn=[w1,w2], 0<w1,w2<1
[b,a]=ellip(n,Rp,Rs,wn); %BPF: wn=[w1,w2], 0<w1,w2<1
```

Rp: Passband ripple in dB

Rs: Minimum stopband attenuation in dB

b: Numerator of transfer function

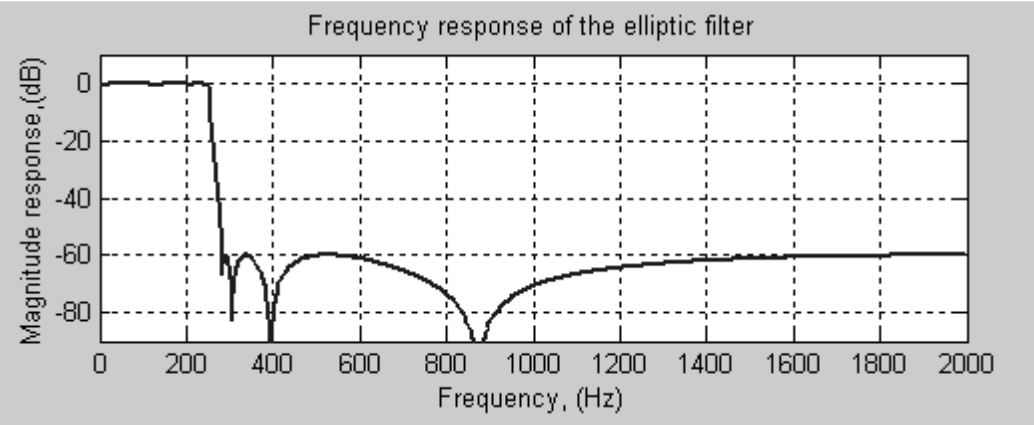
a: denominator of transfer function

**Example 2:**

Design an 8-th order elliptic lowpass filter with a cutoff frequency of 250 Hz, 0.5 dB of ripple in the passband and a minimum stopband attenuation of 60 dB to be used with a signal that is sampled at 4 kHz.

```
clear all;
Fs=4000;
Rp=0.5; Rs=60;
Fn=Fs/2;
Wc=250/Fn;
[b,a]=ellip(8,Rp,Rs, Wc);
[H,f]=freqz(b,a,512,4000);
mag_dB=20*log10(abs(H));
```

```
plot(f,mag_dB,'LineWidth',2); grid
axis([0 2000 -90 10])
xlabel('Frequency, (Hz)')
ylabel('Magnitude response, (dB)')
title('Frequency response of the elliptic filter')
```




**Example 3:**

Suppose you have a 10 kHz sampling frequency and want a bandpass filter with a passband from 1000 to 2000 Hz, lower stopband up to 500Hz and upper stopband starting at 2500 Hz, at most 1 dB of passband ripple, and at least 60 dB of stopband attenuation. You can meet these specifications by using the butter function as follows.

```
[n,Wn] = buttord([1000 2000]/5000,[500 2500]/5000,1,60)
[b,a] = butter(n,Wn);
```

An elliptic filter that meets the same requirements is given by

```
[n,Wn] = ellipord([1000 2000]/5000,[500 2500]/5000,1,60)
[b,a] = ellip(n,1,60,Wn);
```

**MATLAB filter:**

The code **`y=filter(b,a,x);`** filters the data in vector **x** (input) with the filter described by coefficient vectors **a** (denominator) and **b** (numerator) to create the filtered data vector **y** (output). The command `filter` works for both real and complex inputs. If a(1) ≠0, filter normalizes the filter coefficients by a(1). The function **filtfilt** is used in the same way as filter and performs forward-backward filtering (zero-phase filtering).

## Notch Filter

The notch filter is characterized by the sharp attenuation dip in the frequency domain. This filter type is generally used to remove a disturbance of a known narrow-band frequency. The width of the attenuation and the maximum attenuation depends on the order and implementation of the filter.

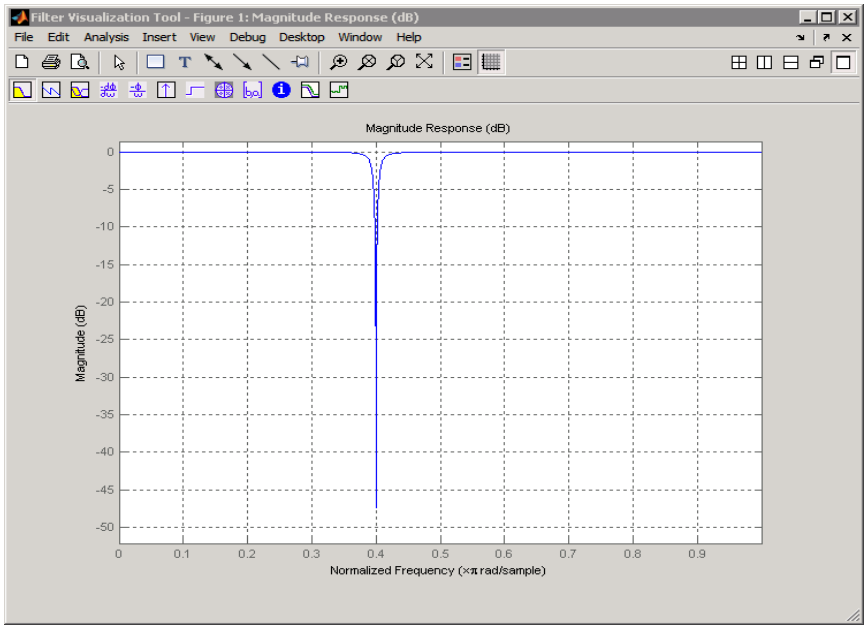
`[b,a] = iirnotch(w0,bw);` returns a digital notching filter with the notch located at w0, and with the bandwidth at the -3 dB point set to bw. To design the filter, w0 must meet the condition 0.0< w0<1.0, where 1.0 corresponds to π radians per sample or $f_s$ Hz in the frequency range. The quality factor (Q factor) q for the filter is related to the filter bandwidth by q= w0/bw .

**Example 4:**

Design and plot an IIR notch filter that removes a 60 Hz tone (f0) from a signal at the sampling frequency of 300 Hz (fs). For this example, set the Q factor for the filter to 35 and use it to specify the filter bandwidth:

```
wo = 60/(300/2);  bw = wo/35;
[b,a] = iirnotch(wo,bw);
fvtool(b,a);
```

As Shown in the output plot of the filter visualization tool, the notch filter has the desired bandwidth with the notch located at 60 Hz, or 0.4π radians per sample.



## De-noising and signal manipulation

### Problem 1:

Use the first stage IIR design routines, buttord, cheby1ord, cheby2ord, and ellipord to find the filter order required for a lowpass filter that attenuates by at least 40 dB. Assume passband and stopband edge frequencies of 100 Hz and 350 Hz respectively, and a sampling frequency of 2 KHz. Furthermore, 1 dB ripple in passband frequencies is acceptable. Compare the differences in order observed for the different filters. Reply the following questions:

a) which filter would be better to use if minimizing the number of operations is one of the aims of your analysis?
b) which filter would be better to use if a steep roll-off is a key requirement?

### Problem 2:

Download the NoisyECG signal from blackboard. The signal has been sampled with a frequency of 500 Hz and the values in the variable ecg are directly in volts. The amplification factor on the electrode signals is 500.

a) You worked on a less noisy version of this signal previously in Lab 2. Just as you did then, start by plotting the signal with the proper units on the axes.

b) Plot the spectrum of the signal with proper units on the frequency axis. Try to identify the power-line noise and body motion artifacts.

c) Design a filter for removing the power line noise as a Notch filter with a zero at 50 Hz. Apply the filter to the signal and generate a new signal ecg2. Plot the new signal and note the difference.

d) The ECG signal is also contaminated with a motion artifact and underlying noise that spans all the spectrum. The amount of noise is proportional to the measurement bandwidth. A higher bandwidth will give more noise in the signals, and limiting the bandwidth can obscure details in the ECG. Design a filtering routine that will remove the motion artifact and the underlying noise up to what is possible. Try to find a compromise on the cut-off frequency for the filter removing the underlying noise, where the shape of the ECG signal is preserved and the noise reduced. Try different choices and plot and comment on the results. Plot the new filtered signal ecg3 and comment on how it is improved over the raw data.

### Problem 3:

In this exercise we will “clean” an EMG signal and design an automatic algorithm for the estimation of the activation timings of the EMG.

In the file 'noisy_EMG.mat' you will find a signal relative to the recording of the EMG signal of the Vastus Medialis muscle during several repetition of a Leg Extension exercise.

The EMG is corrupted by various noises, specifically, a movement artifact noise and the 50 Hz interference. The sampling frequency is 1 KHz.

Keeping in mind that the EMG signal has useful frequency content between about 20 and 450 Hz:

1) Plot the noisy signal in the time domain and its spectrum in the frequency domain

2) Design a filtering routine for cleaning the data, based on the knowledge you gathered so far. Justify your design choices (e.g. why you picked certain cut-off frequencies, why you decided to use a specific filter, why you chose that order for your filter). Keep in mind that you can decide to sacrifice a little bit of information to obtain a better filtering. In this specific case the movement artifact overlaps a little bit with the frequencies of the EMG, then you can decide to keep a little bit of noise or filter a little bit of signal. Plot the resulting "clean" signal in time and its spectrum in the frequency domain.

3) Extract the envelope of the EMG. Check Lecture 9 for the processing steps.

4) Design an algorithm to determine the activation timings of the EMG. Check lecture 9 for guidance. The output of the algorithm will be a vector that will have value 0 when the muscle is not active and 1 when the muscle is active. The decision will be based on a threshold. You can use the first 5 seconds of the signal, where the muscle is not active, to determine the threshold for your algorithm. The threshold will consist of the average + *n**standard deviations of the signal in a point where the muscle is not active.

5) Try different values of *n* until your algorithm returns an activation signal that is equal to 1 only during the 11 contractions that are clearly visible from the raw data. Divide the envelope by its maximum value and plot the envelope and the activation signal on the same plot.

6) Find the activation instants (e.g. the instants where the output of your algorithm goes from 0 to 1). An easy way to do it is to check when the derivative of the 0/1 activation signal is equal to 1. To calculate the derivative of a signal, use the command *diff*

# Upload the report and code on Brightspace by November 16 at 11 pm